

Musichords

Modelagem de Progressões Harmônicas para Guitarra via Grafos de Campos Harmônicos

Integrantes:

Gustavo Kiyoshi Ikeda RA: 10439179
Pedro Montarroyos de Pinho RA: 10440213
Guilherme Limeira de Souza RA: 10439777

Sumário

1. Introdução
2. Definição da Demanda
 - 2.1. O problema ou oportunidade percebida
 - 2.2. A razão ou justificativa para esta demanda
 - 2.3. Descrição sucinta do produto de software que será produzido
 - 2.4. Clientes, usuários e demais envolvidos/impactados com o produto
 - 2.5. Principais etapas necessárias para construir este produto
 - 2.6. Principais critérios de qualidade para o produto
3. Requisitos do Produto
 - 3.1 Tabela de Requisitos
 - 3.2 Backlog do Produto
4. Wireframes
5. Arquitetura Leve
 - 5.1 Diagrama de casos
 - 5.2 Diagrama de classes
6. Arquitetura em camadas
 - 6.1 Ferramentas e Tecnologias

1. Introdução

O Musichords é um aplicativo voltado para guitarristas e violonistas iniciantes que têm dificuldade em conectar a teoria musical à prática do instrumento. A maioria de quem começa a tocar aprende acordes de forma isolada, memorizando posições dos dedos no braço sem entender por que certos acordes soam bem em sequência, o que torna o aprendizado mais lento e desmotivador.

A proposta do Musichords é resolver essa lacuna de forma interativa: o usuário seleciona notas diretamente em uma representação do braço da guitarra, e o aplicativo identifica o acorde correspondente, reconhece os papéis que esse acorde pode exercer em diferentes tonalidades e sugere progressões de acordes que sejam, ao mesmo tempo, harmonicamente corretas e fisicamente fáceis de tocar.

Além da recomendação de progressões, o Musichords também pretende funcionar como um material de consulta interativo sobre teoria musical aplicada permitindo ao usuário explorar campos harmônicos, escalas e diferentes posições de um mesmo acorde no braço do instrumento, tornando-se uma espécie de "livro aberto" de harmonia para guitarra.

2. Definição da Demanda

2.1. O problema ou oportunidade percebida

Iniciantes de guitarra e violão costumam aprender acordes de forma isolada — memorizando shapes (posições dos dedos no braço) sem entender por que determinados acordes soam bem em sequência. Essa desconexão entre a teoria harmônica (campos harmônicos, funções tonais) e a prática instrumental é um dos principais obstáculos enfrentados por autodidatas, e é raramente resolvida pelos aplicativos de acordes disponíveis atualmente, que em sua maioria se limitam a exibir diagramas estáticos de shapes, sem relacioná-los entre si nem sugerir progressões musicalmente válidas.

A oportunidade percebida é construir uma ferramenta que feche essa lacuna: a partir de um acorde tocado pelo usuário no braço do instrumento, o sistema identifica esse acorde, mapeia seus papéis funcionais em diferentes tonalidades e sugere progressões de acordes que sejam, ao mesmo tempo, harmonicamente corretas e fisicamente fáceis de executar

2.2. A razão ou justificativa para esta demanda

A maior parte dos aplicativos de acordes para guitarra resolve apenas parte do problema: mostram como tocar um acorde isolado, mas não indicam quais acordes seguem bem harmonicamente, nem levam em conta o custo físico real de alternar entre eles no braço do instrumento. Isso força o aluno a aprender teoria musical e técnica instrumental como disciplinas separadas, prolongando a curva de aprendizado e desmotivando iniciantes.

Existe, portanto, uma demanda real por uma ferramenta que una as duas pontas teoria e prática em uma única experiência interativa, recomendando progressões que sejam

simultaneamente corretas em teoria musical e viáveis de tocar, otimizando o aprendizado tanto do ponto de vista conceitual quanto motor.

2.3. Descrição sucinta do produto de software que será produzido

O Musichords é um aplicativo no qual o usuário seleciona notas diretamente em uma representação interativa do braço da guitarra. A partir dessa seleção, o sistema identifica o acorde correspondente e, utilizando um grafo direcionado e ponderado que modela os campos harmônicos das tonalidades, mapeia os papéis funcionais desse acorde (tônica, subdominante, dominante etc.) em diferentes tonalidades relacionadas.

Com base nesse mapeamento, o aplicativo sugere progressões de acordes por meio de um algoritmo de busca de caminho mínimo sobre o grafo, as sequências que exigem o menor esforço físico de transição entre os shapes no braço do instrumento. Em uma visão de produto mais ampla, o Musichords também pretende funcionar como um material de consulta interativo ("livro aberto") sobre campos harmônicos, escalas e posições de acordes, oferecendo múltiplas formas de uso além da recomendação de progressões.

2.4. Clientes, usuários e demais envolvidos/impactados com o produto

- Usuários finais: guitarristas e violonistas iniciantes ou autodidatas, que utilizam o aplicativo para aprender progressões de acordes de forma guiada.
- Usuários intermediários/avançados: músicos que desejam usar o aplicativo como ferramenta de consulta sobre campos harmônicos, escalas e variações de shapes (camada de conteúdo mais aprofundado).
- Clientes potenciais: escolas de música e professores particulares de violão/guitarra, que podem adotar o aplicativo como material de apoio didático.

2.5. Principais etapas necessárias para construir este produto

1. Levantamento e priorização de requisitos, organização do backlog do produto.
2. Modelagem e construção da base de dados do grafo de campos harmônicos vértices, arestas funcionais e de pivô, pesos baseados no esforço físico de transição entre shapes.
3. Implementação do módulo de identificação de acordes a partir das notas selecionadas pelo usuário.
4. Implementação do algoritmo de busca de progressões (caminho mínimo) sobre o grafo de campos harmônicos.
5. Implementação da camada de exibição dos shapes físicos correspondentes à progressão escolhida.
6. Prototipagem da interface (wireframes), com foco na representação interativa do braço do instrumento e na exibição das progressões sugeridas.

2.6. Principais critérios de qualidade para o produto

- Corretude musical: as progressões sugeridas pelo sistema devem ser sempre harmonicamente válidas, refletindo corretamente as regras de campos harmônicos utilizados na modelagem.
- Confiabilidade dos dados: a base de acordes, shapes e pesos das arestas deve refletir digitações reais e fisicamente corretas no braço do instrumento.

- Extensibilidade: a arquitetura do grafo deve permitir a adição futura de novas tonalidades, escalas e camadas, sem necessidade de redesenhar o modelo.
- Usabilidade: a interface deve ser intuitiva o suficiente para um usuário iniciante, sem conhecimento prévio de teoria musical, conseguir utilizar o aplicativo sem dificuldade.

3. Requisitos do Produto

3.1. Tabela de requisitos

ID	Tipo de Requisito	Descrição	Prioridade
RF01	Funcional	O sistema deve mapear os papéis funcionais (graus) que o acorde identificado exerce nas diferentes tonalidades do modelo.	Alta
RF02	Funcional	O sistema deve calcular a progressão de menor custo físico de execução, usando busca de caminho mínimo sobre o grafo de campos harmônicos.	Alta
RF03	Funcional	O sistema deve permitir que o usuário selecione notas em uma representação interativa do braço do violão/guitarra.	Média
RF04	Funcional	O sistema deve identificar o acorde correspondente às notas selecionadas pelo usuário.	Alta
RF05	Funcional	O sistema deve permitir ao usuário consultar informações teóricas (campo harmônico, escalas, notas do acorde) associadas a uma nota escolhida.	Alta
RF06	Funcional	O sistema deve sugerir progressões de acordes harmonicamente válidas a partir do acorde inicial.	Alta
RF07	Funcional	O sistema deve exibir visualmente os shapes (posições no braço) de cada acorde da progressão sugerida.	Alta
RNF01	Não Funcional	O sistema deve responder à busca de progressões em tempo real	Alta
RNF02	Não Funcional	A base de dados de shapes e pesos das arestas deve refletir digitações reais e fisicamente corretas no braço do instrumento.	Alta
RNF03	Não Funcional	A arquitetura do grafo deve ser extensível, permitindo a adição futura de novas tonalidades e escalas sem redesenho do modelo.	Alta
RNF04	Não Funcional	A interface deve ser intuitiva para usuários sem conhecimento prévio de teoria musical.	Média

3.2. Backlog do produto

História do Usuário	Requisitos	Prioridade
O usuário iniciante, seleciona as notas no braço interativo para que o sistema identifique o acorde que está tocando	RF03, RF07, RNF01	Alta
O sistema deve reconhecer os papéis funcionais do acorde identificado em diferentes tonalidades, para entender onde ele se encaixa harmonicamente	RF04	Alta
O sistema deve usar uma base de digitações fisicamente corretas, para que as sugestões sejam realmente intocáveis no instrumento	RF01, RF02	Alta
O usuário deverá receber uma progressão de acordes harmonicamente válida a partir do acorde tocado, para aprender sequências corretas sem precisar saber teoria musical avançada	RF06, RNF02	Alta
A progressão sugerida deve priorizar os acordes fisicamente mais fáceis de encadear, para reduzir o esforço de trocar de posição no braço	RF07, RNF02	Alta
O sistema deverá exibir os shapes de cada acorde da progressão sugerida, para saber exatamente onde colocar os dedos	RF05	Média

4. Wireframes

WIREFRAME - FIDELIDADE DE ESTRUTURA, NÃO VISUAL FINAL

Musichords — Fluxo do Aplicativo

Protótipo interativo do fluxo descrito na simulação do projeto: do toque no braço até a progressão sugerida.

PASSO 1

Toque no braço da guitarra

	solta	1	2	3	4
E	E	F	F#	G	G#
B	B	C	C#	D	D#
G	G	G#	A	A#	B
D	D	D#	E	F	F#
A	A	A#	B	C	C#
E	E	F	F#	G	G#

Toque 3 notas para formar um acorde. O sistema reconhece Sol, Dó, Ré, Mi menor, Lá menor e Si menor neste protótipo.

Limpar

Identificar acorde

O QUE ESTÁ ACONTECENDO

Passo 1/6

O usuário toca as notas de um acorde diretamente no braço interativo. Cada círculo corresponde a uma posição real no instrumento.

1. Tocar no braço
2. Identificar acorde
3. Mapear no grafo
4. Progressões por dificuldade
5. Ver shapes
6. Extras

WIREFRAME - FIDELIDADE DE ESTRUTURA, NÃO VISUAL FINAL

Musichords — Fluxo do Aplicativo

Protótipo interativo do fluxo descrito na simulação do projeto: do toque no braço até a progressão sugerida.

PASSO 1

Toque no braço da guitarra

	solta	1	2	3	4
E	E	F	F#	G	G#
B	B	C	C#	D	D#
G	G	G#	A	A#	B
D	D	D#	E	F	F#
A	A	A#	B	C	C#
E	E	F	F#	G	G#

Toque 3 notas para formar um acorde. O sistema reconhece Sol, Dó, Ré, Mi menor, Lá menor e Si menor neste protótipo.

Limpar

Identificar acorde

O QUE ESTÁ ACONTECENDO

Passo 1/6

O usuário toca as notas de um acorde diretamente no braço interativo. Cada círculo corresponde a uma posição real no instrumento.

1. Tocar no braço
2. Identificar acorde
3. Mapear no grafo
4. Progressões por dificuldade
5. Ver shapes
6. Extras

WIREFRAME · FIDELIDADE DE ESTRUTURA, NÃO VISUAL FINAL

Musichords — Fluxo do Aplicativo

Protótipo interativo do fluxo descrito na simulação do projeto: do toque no braço até a progressão sugerida.

PASSO 2

Identificação do acorde

Notas selecionadas: {G, C, E}

Dó Maior

Voltar

Avançar

O QUE ESTÁ ACONTECENDO

Passo 2/6

O sistema identifica o acorde formado pelas notas selecionadas, validando-o contra o campo harmônico conhecido.

1. Tocar no braço
2. Identificar acorde
3. Mapear no grafo
4. Progressões por dificuldade
5. Ver shapes
6. Extras

WIREFRAME · FIDELIDADE DE ESTRUTURA, NÃO VISUAL FINAL

Musichords — Fluxo do Aplicativo

Protótipo interativo do fluxo descrito na simulação do projeto: do toque no braço até a progressão sugerida.

PASSO 3

Mapeamento em grafos de campos harmônicos

CASO 1 — TÔNICA
Grau I de Dó Maior

CASO 2 — SUBDOMINANTE
Grau IV de Sol Maior

CASO 3 — DOMINANTE
Grau V de Fá Maior

acorde físico → múltiplos vértices no grafo

Voltar

Avançar

O QUE ESTÁ ACONTECENDO

Passo 3/6

O mesmo acorde físico pode ocupar mais de um vértice no grafo — cada um representando um papel funcional diferente (tônica, subdominante, dominante...).

1. Tocar no braço
2. Identificar acorde
3. Mapear no grafo
4. Progressões por dificuldade
5. Ver shapes
6. Extras

PASSO 4

Progressões sugeridas

Ordenadas pelo caminho mínimo de alteração dos dedos entre um acorde e o próximo — da mais fácil para a mais difícil de tocar.

Mi menor – Dó – Sol – Ré Mais fácil

$D^{\flat} - 4^{\circ} - 1^{\circ} - 6^{\circ}$ - custo físico total: 13

A progressão só rotacionada a partir do Mi menor. Ganha um tom introspectivo e dramático.

Sol – Ré – Mi menor – Dó custo 14

$1^{\circ} - 6^{\circ} - 3^{\circ} - 4^{\circ}$ - custo físico total: 14

A famosa “progressão pop”. Passa pelo Mi menor, mas soa épica, otimista e triunfante.

Lá menor – Mi menor – Si menor – Dó

custo 14

$2^{\circ} - 4^{\circ} - 3^{\circ} - 1^{\circ}$ - custo físico total: 14

Um caminho denso e melancólico, que transita quase inteiramente pelos graus menores.

Sol – Dó – Ré – Sol custo 15

$1^{\circ} - 4^{\circ} - 5^{\circ} - 1^{\circ}$ - custo físico total: 15

A progressão mais clássica e resolutive possível. Utiliza puramente os pilares maiores do campo harmônico.

[Voltar](#)

[Avançar](#)

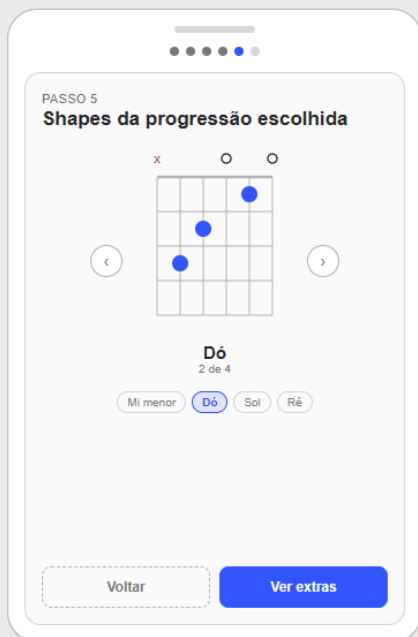
1. Tocar no braço
2. Identificar acorde
3. Mapear no grafo
- 4. Progressões por dificuldade**
5. Ver shapes
6. Extras

1. Tocar no braço
2. Identificar acorde
3. Mapear no grafo
4. Progressões por dificuldade
- 5. Ver shapes**
6. Extras

WIREFRAME · FIDELIDADE DE ESTRUTURA, NÃO VISUAL FINAL

Musichords — Fluxo do Aplicativo

Protótipo interativo do fluxo descrito na simulação do projeto: do toque no braço até a progressão sugerida.



O QUE ESTÁ ACONTECENDO

Passo 5/6

A progressão escolhida é exibida acorde a acorde, com o shape físico correspondente no braço do instrumento.

1. Tocar no braço
2. Identificar acorde
3. Mapear no grafo
4. Progressões por dificuldade

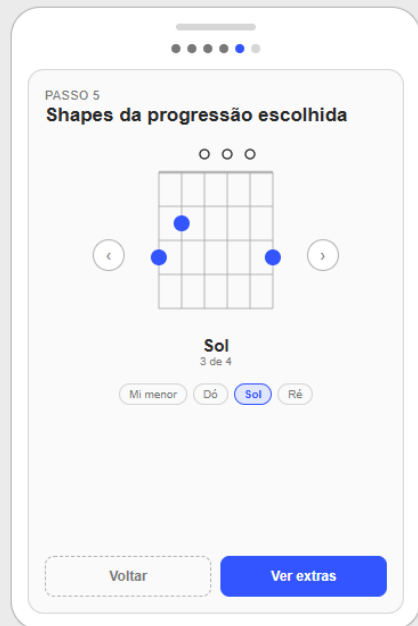
5. Ver shapes

6. Extras

WIREFRAME · FIDELIDADE DE ESTRUTURA, NÃO VISUAL FINAL

Musichords — Fluxo do Aplicativo

Protótipo interativo do fluxo descrito na simulação do projeto: do toque no braço até a progressão sugerida.



O QUE ESTÁ ACONTECENDO

Passo 5/6

A progressão escolhida é exibida acorde a acorde, com o shape físico correspondente no braço do instrumento.

1. Tocar no braço
2. Identificar acorde
3. Mapear no grafo
4. Progressões por dificuldade

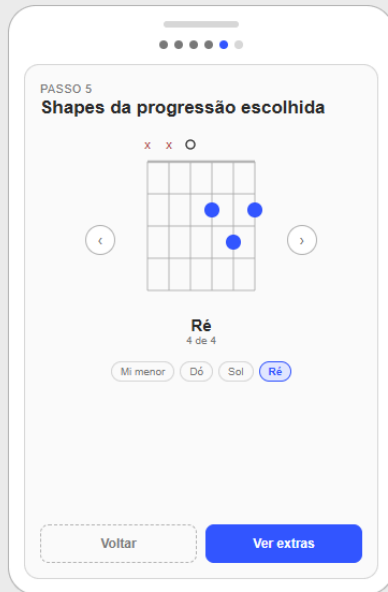
5. Ver shapes

6. Extras

WIREFRAME - FIDELIDADE DE ESTRUTURA, NÃO VISUAL FINAL

Musichords — Fluxo do Aplicativo

Protótipo interativo do fluxo descrito na simulação do projeto: do toque no braço até a progressão sugerida.



O QUE ESTÁ ACONTECENDO

Passo 5/6

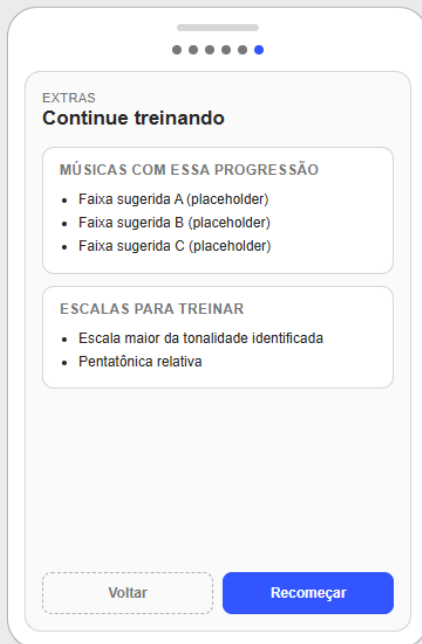
A progressão escolhida é exibida acorde a acorde, com o shape físico correspondente no braço do instrumento.

1. Tocar no braço
2. Identificar acorde
3. Mapear no grafo
4. Progressões por dificuldade
5. **Ver shapes**
6. Extras

WIREFRAME - FIDELIDADE DE ESTRUTURA, NÃO VISUAL FINAL

Musichords — Fluxo do Aplicativo

Protótipo interativo do fluxo descrito na simulação do projeto: do toque no braço até a progressão sugerida.



O QUE ESTÁ ACONTECENDO

Passo 6/6

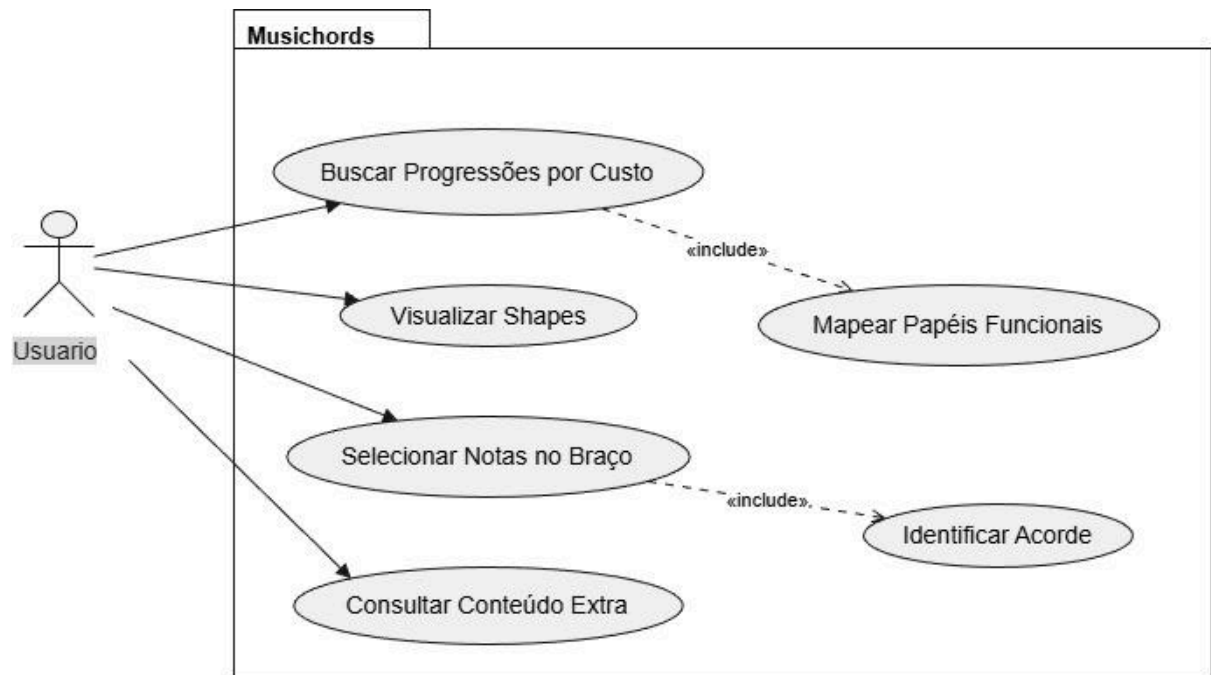
Conteúdo complementar: músicas de referência e escalas relacionadas para o usuário continuar praticando.

1. Tocar no braço
2. Identificar acorde
3. Mapear no grafo
4. Progressões por dificuldade
5. Ver shapes
6. **Extras**

5. Arquitetura Leve

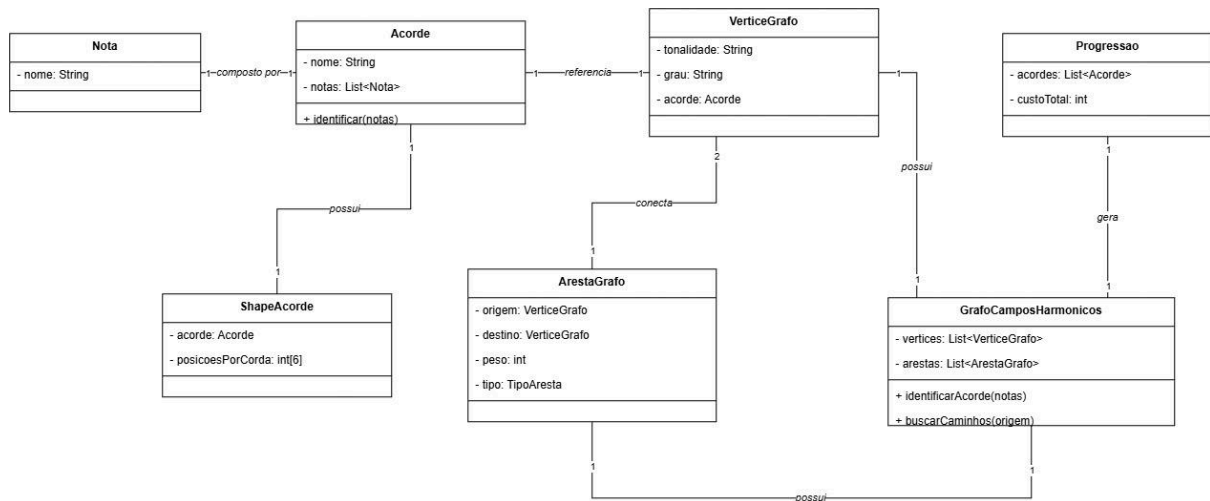
5.1. Diagrama de casos de uso

O sistema possui um único ator principal, o Usuário, responsável por todas as interações do fluxo. Os casos de uso seguem a ordem natural de uso do aplicativo: da seleção das notas no braço até a visualização dos shapes da progressão escolhida.



5.2. Diagrama de classes (modelo de domínio)

O diagrama a seguir representa as principais entidades do domínio do problema e como elas se relacionam, servindo de base conceitual para o modelo do grafo de campos harmônicos descrito nas seções anteriores. Por se tratar de uma modelagem leve, o diagrama omite detalhes de visibilidade e métodos auxiliares, focando nos atributos e relacionamentos essenciais para o entendimento do sistema.

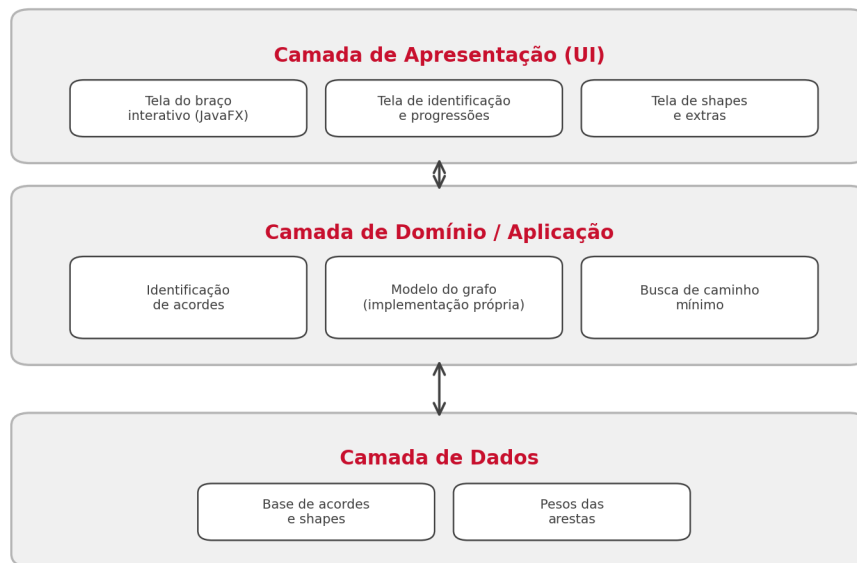


Um acorde é composto por uma ou mais Notas e pode referenciar múltiplos VerticeGrafo, já que o mesmo acorde físico ocupa papéis funcionais diferentes em tonalidades distintas (o mecanismo de pivô já discutido na modelagem do grafo). Cada VerticeGrafo se conecta a outros por meio de ArestaGrafo, cujo atributo "tipo" distingue arestas funcionais (dentro de uma tonalidade) de arestas de pivô (entre tonalidades, custo zero). A classe GrafoCamposHarmonicos concentra a lógica de identificação de acordes e busca de progressões, gerando objetos Progressao ordenados por custo total.

6. Arquitetura em camadas

O Musichords será desenvolvido como um aplicativo desktop, estruturado em três camadas, separando claramente a interface do usuário, a lógica de negócio (modelagem do grafo e algoritmos de busca) e o armazenamento dos dados. Essa separação facilita a manutenção do projeto e permite testar a lógica de identificação de acordes e de busca de progressões de forma independente da interface gráfica.

Musichords — Arquitetura em Camadas



Camada de Apresentação (UI): concentra as telas do fluxo do aplicativo — seleção de notas no braço interativo, identificação do acorde, exibição dos papéis funcionais no grafo, lista de progressões ordenadas por dificuldade e exibição dos shapes da progressão escolhida.

Camada de Domínio/Aplicação: contém as regras centrais do sistema — reconhecimento de acordes a partir das notas selecionadas, o modelo do grafo de campos harmônicos (vértices representando o papel funcional de cada acorde em cada tonalidade, arestas funcionais e de pivô) e o algoritmo de busca de caminho mínimo, implementados pelo próprio grupo, usados para ordenar as progressões sugeridas pelo esforço físico de execução.

Camada de Dados: responsável por armazenar e fornecer a base de acordes, os shapes (digitações) de cada acorde e os pesos das arestas do grafo, que alimentam os cálculos realizados pela camada de domínio.

6.1 Ferramentas e tecnologias

A tabela a seguir resume as principais tecnologias selecionadas para o desenvolvimento do Musichords como aplicativo desktop. A modelagem do grafo de campos harmônicos e o algoritmo de busca de caminho mínimo serão implementados pelo próprio grupo, sem uso de bibliotecas prontas de grafos.

Linguagem	Java
Interface (UI)	JavaFX
Modelagem do grafo	Implementação própria (estruturas de dados e algoritmo de busca de caminho mínimo desenvolvidos pelo grupo, sem bibliotecas prontas)

Versionamento	Git / GitHub
Testes	JUnit
Metodologia	Scrum, sprints semanais